

CS-4112: Introduction to Machine Learning

Backpropagation and Vanishing Gradients

Arbish Akram

Department of Computer Science
Government College University

Slides by Dr. Nazar Khan

Backpropagation

Learning Algorithm

1. Forward propagate the input vector $\mathbf{x}_n$ to compute *and store* activations and outputs of every neuron in every layer.
2. Evaluate $\delta_k = \frac{\partial L_n}{\partial a_k}$ for every neuron in output layer.
3. Evaluate $\delta_j = \frac{\partial L_n}{\partial a_j}$ for every neuron in *every* hidden layer via backpropagation.

$$\delta_j = h'(a_j) \sum_{k=1}^K \delta_k w_{kj}$$

4. Compute derivative of each weight $\frac{\partial L_n}{\partial w}$ via $\delta \times \text{input}$.
5. Update each weight via gradient descent $w^{\tau+1} = w^{\tau} - \eta \frac{\partial L_n}{\partial w}$.

Tanh

$A(-1, 1)$ sigmoidal function

- ▶ Since range of logistic sigmoid $\sigma(a)$ is $(0, 1)$, we can obtain a function with $(-1, 1)$ range as $2\sigma(a) - 1$.
- ▶ Another related function with $(-1, 1)$ range is the **tanh** function.

$$\tanh(a) = 2\sigma(2a) - 1 = \frac{e^a - e^{-a}}{e^a + e^{-a}}$$

where σ is applied on $2a$.

- ▶ Preferred¹ over logistic sigmoid as activation function $h(a)$ of hidden neurons.
- ▶ Just like the logistic sigmoid, derivative of $\tanh(a)$ is simple:
 $1 - \tanh^2(a)$. (Prove it.)

¹lecun1998efficient.

A Simple Example

- ▶ Two-layer MLP for multivariate regression from $\mathbb{R}^D \rightarrow \mathbb{R}^K$.
- ▶ Linear outputs $y_k = a_k$ with half-SSE $L = \frac{1}{2} \sum_{k=1}^K (y_k - t_k)^2$.
- ▶ M hidden neurons with $\tanh(\cdot)$ activation functions.

Forward propagation

$$a_j = \sum_{i=0}^D w_{ji}^{(1)} x_i$$

$$z_j = \tanh(a_j)$$

$$z_0 = 1$$

$$y_k = \sum_{j=0}^M w_{kj}^{(2)} z_j$$

$$\delta_k = y_k - t_k$$

- ▶ Compute derivatives $\frac{\partial L}{\partial w_{ji}^{(1)}} = \delta_j x_i$ and $\frac{\partial L}{\partial w_{kj}^{(2)}} = \delta_k z_j$.

Backpropagation

$$\delta_j = (1 - z_j^2) \sum_{k=1}^K w_{kj}^{(2)} \delta_k$$

Minimization via Gradient Descent

- ▶ To minimize loss $L(\mathbf{w})$ with respect to weights $\mathbf{w}$

$$\mathbf{w}^{\text{new}} = \mathbf{w}^{\text{old}} - \eta \nabla_{\mathbf{w}} L(\mathbf{w})$$

where scalar $\eta > 0$ controls the step-size. It is called the *learning rate*.

- ▶ Also known as *gradient descent*.

Repeated applications of gradient descent find the closest local minimum.

Gradient Descent

1. Initialize $\mathbf{w}^{\text{old}}$ randomly.
2. do
 - 2.1 $\mathbf{w}^{\text{new}} \leftarrow \mathbf{w}^{\text{old}} - \eta \nabla_{\mathbf{w}} L(\mathbf{w})|_{\mathbf{w}^{\text{old}}}$
3. while $|L(\mathbf{w}^{\text{new}}) - L(\mathbf{w}^{\text{old}})| > \epsilon$

- ▶ Learning rate η needs to be reduced gradually to ensure *convergence to a local minimum*.
- ▶ If η is too large, the algorithm can *overshoot* the local minimum and keep doing that indefinitely (*oscillation*).
- ▶ If η is too small, the algorithm will take too long to reach a local minimum.

Gradient Descent

- ▶ Different types of gradient descent:

Batch $\mathbf{w}^{\text{new}} = \mathbf{w}^{\text{old}} - \eta \nabla_{\mathbf{w}} L$

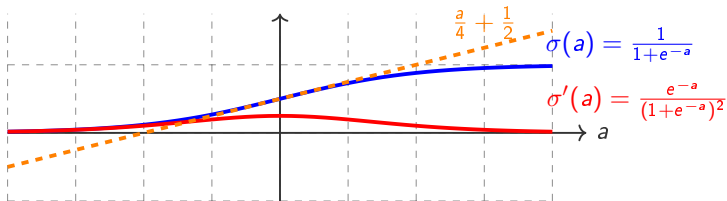
Sequential $\mathbf{w}^{\text{new}} = \mathbf{w}^{\text{old}} - \eta \nabla_{\mathbf{w}} L_n$

Stochastic same as sequential but n is chosen randomly

Mini-batches $\mathbf{w}^{\text{new}} = \mathbf{w}^{\text{old}} - \eta \nabla_{\mathbf{w}} L_{\mathcal{B}}$

- ▶ Most common variations are stochastic gradient descent (SGD) and SGD using mini-batches.

Problems with sigmoidal neurons



- ▶ For large $|a|$, sigmoid value approaches either 0 or 1. This is called *saturation*.
- ▶ When the sigmoid saturates, the gradient approaches zero.
- ▶ Neurons with sigmoidal activations stop learning when they saturate.
- ▶ When they are not saturated, they are *almost linear*.
- ▶ There is another reason for the gradient to approach zero during backpropagation.

Vanishing Gradient

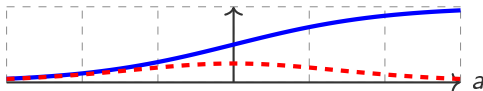
- ▶ Notice that gradient of the sigmoid is always between 0 and $\frac{1}{4}$.
- ▶ Now consider the backpropagation equation.

$$\delta_j = \underbrace{h'(a_j)}_{\leq \frac{1}{4}} \sum_{k=1}^K w_{kj} \delta_k$$

where δ_k will also contain *at least* one factor of $\leq \frac{1}{4}$.

- ▶ This means that values of δ_j keep getting smaller as we backpropagate towards the early layers.
- ▶ Since gradient = $\delta \times$ input, the gradients also keep getting smaller for the earlier layers. Known as the *vanishing gradient* problem.
- ▶ *Therefore, while the network might be deep, learning will not be deep.*

Logistic Sigmoid



Activation function

$$y(a) = \frac{1}{1+e^{-a}}$$

Derivative

$$y'(a) = y(a)(1 - y(a))$$

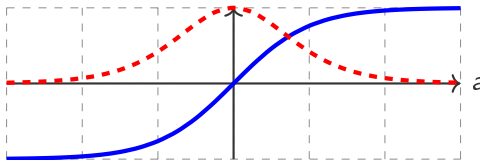
Maximum magnitude of derivative

$$\frac{1}{4}$$

Problem

Cause vanishing gradients

Hyperbolic Tangent



Activation function

$$y(a) = \tanh(a)$$

Derivative

$$y'(a) = 1 - y^2(a)$$

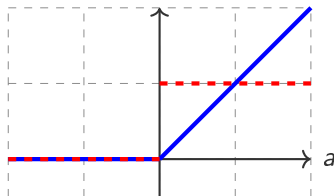
Maximum magnitude of derivative

1

Problem

Cause vanishing gradients

Rectified Linear Unit (ReLU)



Activation function $y(a) = \begin{cases} a & \text{if } a > 0 \\ 0 & \text{if } a \leq 0 \end{cases}$

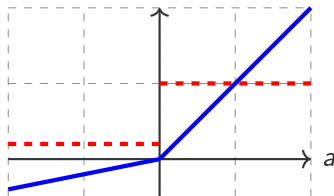
Derivative $y'(a) = \begin{cases} 1 & \text{if } a > 0 \\ 0 & \text{if } a \leq 0 \end{cases}$

Advantage Avoids vanishing gradients

Problem Dead neurons²

²This can be an advantage as well since death implies fewer neurons.

Leaky ReLU



Activation function

$$y(a) = \begin{cases} a & \text{if } a > 0 \\ ka & \text{if } a \leq 0 \end{cases}$$

where $0 \leq k \leq 1$

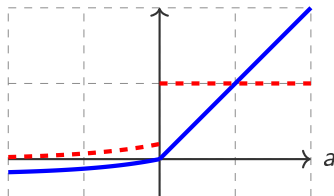
Derivative

$$y'(a) = \begin{cases} 1 & \text{if } a > 0 \\ k & \text{if } a \leq 0 \end{cases}$$

Advantage

Neuron is always learning

Exponential Linear Unit (ELU)



Activation function

$$y(a) = \begin{cases} a & \text{if } a > 0 \\ k(e^a - 1) & \text{if } a \leq 0 \end{cases}$$

where $k > 0$

Derivative

$$y'(a) = \begin{cases} 1 & \text{if } a > 0 \\ y(a) + k & \text{if } a \leq 0 \end{cases}$$

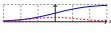
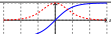
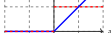
Maximum magnitude of derivative 1

Advantage

Neuron is mostly learning

Activation Functions

Summary

Name	$y(a)$	Plot	$y'(a)$	Comments
Logistic sigmoid	$\frac{1}{1+e^{-a}}$		$y(a)(1 - y(a))$	Vanishing gradients
Hyperbolic tangent	$\tanh(a)$		$1 - y^2(a)$	Vanishing gradients
Rectified Linear Unit (ReLU)	$\begin{cases} a & \text{if } a > 0 \\ 0 & \text{if } a \leq 0 \end{cases}$		$\begin{cases} 1 \\ 0 \end{cases}$	Dead neurons. Sparsity.
Leaky ReLU	$\begin{cases} a & \text{if } a > 0 \\ ka & \text{if } a \leq 0 \end{cases}$		$\begin{cases} 1 \\ k \end{cases}$	$0 < k < 1$
Exponential Linear Unit (ELU)	$\begin{cases} a & \text{if } a > 0 \\ k(e^a - 1) & \text{if } a \leq 0 \end{cases}$		$\begin{cases} 1 \\ y(a) + k \end{cases}$	$k > 0.$

- Saturated sigmoidal neurons stop learning. Piecewise-linear units keep learning by avoiding saturation.
- ELU has been shown to lead to better accuracy and faster training.
- *Take home message:* For hidden neurons, use a member of the LU family. They avoid *i)* saturation and *ii)* the vanishing gradient problem.